Continuous Co-Occurrence Query over Object Streams from Videos (long version)

Zhongshuai Zhang[1], Jiadong Zhang[2], Baihua Zheng[3], Bin Wang[2], Xiaochun Yang[2]
Beijing Institute of Technology[1], Northeastern University[2], Singapore Management University[3]
zhang_zhongshuai@126.com, jiadong@stumail.neu.edu.cn, bhzheng@smu.edu.sg, {binwang, yangxc}@mail.neu.edu.cn

***Abstract*—Over an object stream extracted from a video feed, a continuous co-occurrence query reports, at every frame, all maximal co-occurrence patterns of a sliding window with at least a given number of objects (size threshold) and at least a given number of frames (duration threshold), where a pattern is a group of objects with all the frames in which they co-occur. The state-of-the-art methods scan frame by frame and derive new states from stored ones, so they cannot prune by duration threshold and must maintain all maximal co-occurrence patterns of the window, which far outnumber the answers. We propose OWIM (Object-Wise Incremental Maintenance), which scans object by object: since the frames in which a group co-occurs only shrink as objects are added, a candidate is discarded as soon as it has too few frames or cannot reach enough objects, and we prove that this pruning remains safe as the window slides. At each frame, OWIM only trims the entries that contain the expiring frame and recomputes the answers that contain the new frame, so its cost is independent of the number of maximal co-occurrence patterns. On real and synthetic videos, OWIM is 64 to 4,055 times faster than the state of the art and 200 to 2,342 times faster on long videos, with a 99th-percentile latency of at most 53 microseconds per frame.**

***Index Terms*—continuous queries, co-occurrence patterns, video analytics, sliding windows, incremental maintenance**

I. Introduction

The volume of video data is growing rapidly, with large numbers of cameras deployed in transportation systems, city streets and shopping malls, and along roads. Queries that retrieve video content matching specific patterns, such as object co-occurrence patterns [1]–[3] and co-movement patterns [4], [5], are key to extracting value from this data. Advances in object detection and multi-object tracking make these queries possible, because they make it practical to convert videos into structured data automatically: real-time detectors [6], [7] recognize the objects in every frame, and tracking-by-detection methods [8]–[10] assign each object an identifier that persists across frames. A video feed can thus be turned into a structured *object stream*, in which each frame gives the set of tracked objects in view.

The *continuous co-occurrence query* $CQ(w, d, k)$ [1] is a fundamental query over object streams: it asks *which objects appear together, and for how long*. Over a sliding window of the w most recent frames, it reports at every frame all *maximal co-occurrence patterns* (MCPs) with at least k objects and at least d frames. An MCP (X, S) is a group of objects X together with all the frames S in which they co-occur, such that neither the group X nor its frames S can be extended.

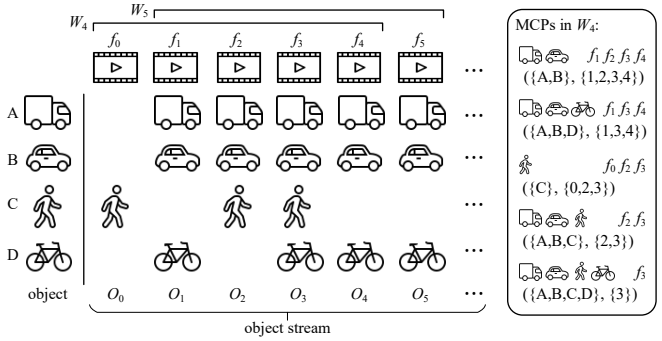


Fig. 1. A running example of $CQ(w = 5, d = 3, k = 3)$.

With $d < w$, the frames of an MCP need not be consecutive, so the query tolerates occlusions and missed detections. Fig. 1 gives an example. Detecting and tracking the objects in each frame yields the object stream $\langle O_0 = \{C\}, O_1 = \{A, B, D\}, O_2 = \{A, B, C\}, O_3 = \{A, B, C, D\}, \dots \rangle$, where O_t is the set of objects appearing in frame f_t. The query $CQ(w = 5, d = 3, k = 3)$ asks, in each window of $w = 5$ frames, for the MCPs in which at least $k = 3$ objects appear together in at least $d = 3$ frames. At $t = 4$, the window W_4 (frames 0–4) contains five MCPs: $(\{A, B\}, \{1, 2, 3, 4\})$, $(\{A, B, D\}, \{1, 3, 4\})$, $(\{C\}, \{0, 2, 3\})$, $(\{A, B, C\}, \{2, 3\})$, and $(\{A, B, C, D\}, \{3\})$. Only $(\{A, B, D\}, \{1, 3, 4\})$ is an answer; $(\{A, B\}, \{1, 2, 3, 4\})$ and $(\{C\}, \{0, 2, 3\})$ have fewer than $k = 3$ objects, and $(\{A, B, C\}, \{2, 3\})$ and $(\{A, B, C, D\}, \{3\})$ have fewer than $d = 3$ frames. When frame 5 arrives and frame 0 expires, the answer becomes $(\{A, B, D\}, \{1, 3, 4, 5\})$.

Motivations and Challenges. Many applications depend on analyzing groups of objects. In traffic monitoring, groups of vehicles that travel together, such as platoons, matter for traffic management [4], [5]. In surveillance and public safety, groups of people who stay together in a scene need to be identified [11]. When curating training data for autonomous driving, one needs to check whether the videos contain enough scenes in which several objects appear together, e.g., multiple trucks [1]. All of these applications require results to be produced continuously, frame by frame, over video streams. However, answering continuous co-occurrence queries efficiently faces three challenges. First, the number of MCPs in a window can grow exponentially with the number of objects and it far exceeds the number of answers. W_4 in Fig. 1 has five MCPs but one answer. In our experiments, windows contain

290–6,882 MCPs per frame, yet only 0–4 answers per frame (Section V-B). Second, the results change at every frame as the window slides. When a frame arrives, some MCPs gain a frame and new MCPs emerge. When a frame expires, others lose a frame and may merge with other MCPs. Moreover, occlusions scatter the frames of a group cross the window, so they cannot be summarized by a single interval. Third, query processing is under strict time constraints. Since detection and tracking now run in real time [6]–[9], query maintenance must keep pace with the frame rate, leaving at most a few tens of milliseconds per frame.

State-of-the-Art Efforts. Continuous co-occurrence queries were studied in [1], which introduces two state-maintenance methods, MFS and SSG, to maintain the co-occurrence states. Like an MCP, a state consists of an object set and a frame set, and the valid states are exactly the MCPs of the window. Both methods scan frame by frame (*frame-wise*): the objects of a new frame are intersected with the object sets of existing states to find co-occurrences, and new states are derived from existing ones, inheriting their frames. The appearance history of a group of objects is therefore kept only in its state. A state with fewer than d frames may still reach d as new frames arrive, and discarding it would lose the frames accumulated so far. Hence, states cannot be pruned by d during maintenance. As a result, every MCP in the window is kept as a state, and every new frame visits all existing states, so the cost grows with the total number of MCPs rather than with the size of the answer. In our experiments, SSG maintains 290–6,882 states per frame and takes up to 121 ms to process a single frame of a long video; on dataset D2, state maintenance alone exceeds the 33 ms per frame budget of a 30-fps video on 9% of the frames. To our knowledge, no existing method maintains the answers to continuous co-occurrence queries at a cost independent of the total number of MCPs in the window.

Our Solution. We propose OWIM (Object-Wise Incremental Maintenance), which instead scans *object by object* (*object-wise*). We introduce an appearance frame set for each object o to record the frames in the current window in which this object o appears. Accordingly, the co-occurrence frame set of a group of objects is the intersection of the appearance frame sets of these objects, so it can only shrink as objects are added during the scan. Hence, once a candidate (a group of objects) appears in fewer than d frames, it can be pruned, and so can a candidate that cannot reach k objects. That is, OWIM enables pruning by both d and k. Both pruning rules remain safe as the window slides: every answer that does not contain the new frame is stored, possibly trimmed by the expiring frame, and every answer that contains the new frame is recomputed from the objects of that frame, so a pruned group is found again if its objects co-occur again. We prove that the table $\mathcal{D}$ maintained by OWIM, which stores only the MCPs that satisfy both thresholds, is exactly the answer set at every frame.

OWIM records the frames in which each object appears in the window as an *appearance bitmap*. The co-occurrence frame set of a group is then computed by a bitwise AND of its objects' bitmaps, and its size by a population count. OWIM proceeds in three steps. 1) Initialization: when the query starts, OWIM runs an object-wise dynamic program once over the first window to obtain the initial answers. 2) Expiring frame: at every frame, OWIM first handles the expiring frame. Using an index on the oldest frame of each entry, it trims only the entries that contain the expiring frame, drops those left with fewer than d frames, and merges each remaining one into the entry with the same frame set, if any. 3) New frame: OWIM then reruns the dynamic program on the objects of the new frame only, which yields all answers that contain the new frame, and these replace the old entries they supersede. To support these steps, appearance bitmaps use a circular encoding, so sliding the window changes only one bit of each affected bitmap, and the keys of the stored entries remain valid. Overall, frame t takes $O\big(|O_{t-w}| + |O_t| + (C_e + |O_t| \cdot |\mathcal{D}'|) \cdot \lceil w/64 \rceil\big)$ word operations, where O_t and O_{t-w} are the sets of objects in the new and expiring frames, C_e is the number of entries that contain the expiring frame, and $|\mathcal{D}'|$ is the maximum number of candidates held by the dynamic program on the new frame, all of which contain the new frame. The cost is thus independent of the total number of MCPs in the window.

Our contributions are as follows.

- We analyze the limitation of frame-wise incremental maintenance: it cannot prune by the duration threshold d during maintenance and thus maintains far more states than answers (Section II).
- We propose an object-wise dynamic program for computing MCPs, together with pruning rules based on d and k (Section III).
- We propose an object-wise incremental maintenance approach and prove that both pruning rules remain safe as the window slides and that the maintained table equals the answer set at every frame (Section IV).
- We evaluate OWIM on real and synthetic datasets. Under the default parameters, OWIM is 64–4,055× faster than SSG, the state-of-the-art method, on regular videos and 200–2,342× faster on long videos; its 99th-percentile latency per frame is at most 53 μs (Section V).

II. Preliminaries

A. Problem Definition

1) From Video Feeds to Object Streams: We consider a video feed $V = \langle f_0, f_1, f_2, \dots \rangle$, an unbounded sequence of frames captured by a camera. Off-the-shelf object detection and multi-object tracking models (e.g., YOLO [6] and DeepSORT [12]) are applied to each frame: the detector reports the objects, and the tracker assigns every object an identifier that persists across frames, including frames in which the object is temporarily occluded. Improving the accuracy of these models is orthogonal to this paper; we treat their output as given.

Let $\mathcal{ID}$ be the set of object identifiers. After detection and tracking, frame f_t is represented by the set $O_t \subseteq \mathcal{ID}$ of objects visible in it. The input to our problem is therefore the *object stream* $\mathcal{O} = \langle O_0, O_1, O_2, \dots \rangle$, which arrives one frame at a time. Figure 1 shows the object stream of our running example, which we use throughout the paper.

2) Co-occurrence in a Sliding Window: Queries are evaluated over a sliding window of the most recent w frames. At time t, the window is $W_t = \{\max(0, t-w+1), \ldots, t\}$; it advances by one frame whenever a new frame arrives.

For an object o, its *appearance frame set* in the window W_t is $F_t(o) = \{i \in W_t \mid o \in O_i\}$. For a non-empty object set $X \subseteq \mathcal{ID}$, the frames in which all objects of X appear together form its *co-occurrence frame set* $F_t(X) = \bigcap_{o \in X} F_t(o)$.

Occlusions are common in video, e.g., a car may be hidden behind a truck for a few frames and then reappear with the same identifier. We therefore do not require the frames in $F_t(X)$ to be consecutive. Instead, a *duration threshold* d ($1 \leq d \leq w$) states that X must co-occur in at least d frames of the window, i.e., $|F_t(X)| \geq d$. Choosing $d < w$ tolerates up to $w - d$ frames in which some objects of X are occluded or missed by the detector. For instance, with videos at 30 fps, $w = 300$ and $d = 240$ ask for objects that appear together for at least 8 seconds within a 10-second window.

3) Maximal Co-occurrence Patterns: Co-occurrence is highly redundant. If a group of objects X co-occurs in a set of frames S, every non-empty subset of X also co-occurs in (at least) S, and X also co-occurs in every subset of S. Reporting all such combinations would overwhelm users with answers implied by others. We therefore report only maximal patters: each pairs a group of objects with all the frames in which it co-occurs, such that no other object appreas in all of these frames.

Given a frame set $S \subseteq W_t$, let $cover_t(S) = \{o \in \mathcal{ID} \mid S \subseteq F_t(o)\}$ be the objects that appear in every frame of S.

Definition 1 (Maximal co-occurrence pattern)**.** *Given the window W_t, a pair (X, S) with $\emptyset \neq X \subseteq \mathcal{ID}$ and $\emptyset \neq S \subseteq W_t$ is a* maximal co-occurrence pattern *(MCP) if and only if*

1) *$S = F_t(X)$, i.e., S contains every frame of the window in which all objects of X appear; and*
2) *$X = cover_t(S)$, i.e., X consists of exactly the objects that appear in all frames of S.*

Condition 1 says that S is the full co-occurrence frame set of X. Condition 2 additionally requires X to contain every object that appears in all frames of S. Hence, every frame set and every object set belongs to at most one MCP.

For example, with $w = 5$ and $t = 4$ in Figure 1, the five MCPs of $W_4 = \{0, \ldots, 4\}$ are those listed in Section I. The pair $(\{A, D\}, \{1, 3, 4\})$ is not an MCP, since B also appears in frames 1, 3, and 4 (condition 2 fails); likewise, $(\{A, B\}, \{1, 3, 4\})$ is not an MCP, since A and B also co-occur in frame 2 (condition 1 fails).

4) Continuous Co-occurrence Queries: We define the continuous co-occurrence query over an object stream using a sliding-window model.

Definition 2 (Continuous co-occurrence query)**.** *Let $\mathrm{MCP}(W_t)$ denote the set of MCPs in the window W_t. Given an object stream $\mathcal{O}$, a window size w, a duration threshold d ($1 \leq d \leq w$), and a size threshold $k \geq 1$, the* continuous co-occurrence query *$CQ(w, d, k)$ reports, at every time t, the answer set $Ans_t = \{(X, S) \in \mathrm{MCP}(W_t) \mid |S| \geq d \wedge |X| \geq k\}$.*

That is, at every frame we report every MCP of at least k objects that has co-occurred in at least d of the last w frames.

TABLE I
NOTATION

Symbol	Meaning
O_t	set of objects (identifiers) visible in frame f_t
$\mathcal{O}$	object stream $\langle O_0, O_1, O_2, \ldots \rangle$
w	window size (number of frames)
W_t	window at time t: the last w frames
$F_t(o)$	appearance frame set (bitmap) of object o in W_t
$F_t(X)$	co-occurrence frame set of object set X in W_t
$cover_t(S)$	objects that appear in every frame of S
d	duration threshold: minimum number of co-occurring frames
k	size threshold: minimum number of objects
(X, S)	maximal co-occurrence pattern (MCP)
$\mathrm{MCP}(W_t)$	set of MCPs in the window W_t
Ans_t	answer set of $CQ(w, d, k)$ at time t
$\mathcal{D}$	table of entries $S \mapsto X$; equals Ans_t after frame t

B. Recap of Frame-Wise Incremental Maintenance

The work in [1] studies continuous co-occurrence queries as a special case of temporal queries over video feeds. Its state-maintenance component uses two methods, Marked Frame Set (MFS) and Sparse State Graph (SSG), to maintain the co-occurrence states. A state (X, S) pairs an object set X with the frames S of the window in which all its objects in X appear. One state is kept per distinct object set, and the valid states are exactly the MCPs of the window (Definition 1).

States are updated frame by frame. When frame f_t arrives with object set O_t, each existing state (X, S) intersects its X with O_t. If $Y = X \cap O_t \neq \emptyset$, the objects of Y co-occur in every frame of S and in f_t, so $S \cup \{t\}$ is merged into the state of Y. This state is created if it does not yet exist. When $X \subseteq O_t$ (all objects of X appear in f_t), then this state is updated to $(X, S \cup \{t\})$. When $Y = \emptyset$, the state is unaffected. If no existing state has $X \supseteq O_t$, the state $(O_t, \{t\})$ is also created, since the objects of O_t co-occur in f_t but in no earlier frame of the window. Finally, the expiring frame is removed from all states, and states left with no frames are deleted.

This design offers three key advantages. First, states are reused across windows, so processing a new frame requires only one intersection per state instead of recomputing the whole window. Second, MFS marks selected frames so that a state is an MCP if and only if it has at least one marked frame. After a frame expires, a state's object set may stop being maximal if some other object appears in all of its remaining frames. MFS detects such a state without comparing frame sets. For example, in Figure 1 with $w = 5$, f_0 expires when f_5 arrives, and the state $(\{C\}, \{0, 2, 3\})$ becomes $(\{C\}, \{2, 3\})$. Since A and B also appear in f_2 and f_3, the object set $\{C\}$ in this state is no longer maximal, and the state is covered by $(\{A, B, C\}, \{2, 3\})$. In MFS, f_0 is the only marked frame of the state of $\{C\}$, because it is the only frame whose object set is exactly $\{C\}$; once it expires, the state has no marked frame and is removed, without comparing its frames with those

of $(\{A, B, C\}, \{2, 3\})$. Third, SSG organizes the states into a forest in which the object set of each child is contained in that of its parent; once a state has an empty intersection with the new frame, the intersections for all of its descendants are skipped.

However, frame-wise incremental maintenance has to maintain far more states than answers. By Definition 1, an MCP is maximal in both its object set and its frame set. In frame-wise maintenance, the object set of a new state is the intersection of objects in the new frame with those in every existing state, so it is maximal when the state is created; after frames expire, however, the remaining frames of a state may all contain additional objects, so its object set may no longer be maximal, which is what MFS detects. The frame set of a state grows whenever its objects co-occur again in a new frame, so a state with fewer than d frames may still reach d as the window slides. Hence, the condition $|S| \geq d$ can be checked only when answers are reported and cannot be used to prune states during maintenance. In other words, every MCP of the window has to be maintained regardless of d. The per-frame cost thus grows with the total number of MCPs, which can exceed the size of the answer set by orders of magnitude.

To reduce this cost, we propose *Object-Wise Incremental Maintenance (OWIM)*. Here, the co-occurrence frame set of a group of objects is computed by intersecting the appearance frame sets of its objects, so it is maximal by construction and never grows as more objects are added. This enables pruning by the duration threshold: since every answer satisfies $|S| \geq d$, a candidate with fewer than d frames can be discarded as soon as it is generated. We can also prune by the size threshold: since every answer satisfies $|X| \geq k$, a candidate is discarded during object-wise scanning if it cannot reach k objects even with all remaining objects added. This contrasts with frame-wise maintenance, where every frame visits all MCPs of the window; the larger d and k are, the fewer candidates the object-wise approach generates and keeps.

III. Object-wise Computation of MCPs

This section shows how to compute the answer set Ans_t of a single, fixed window W_t by scanning objects rather than frames. We introduce appearance bitmaps that record where objects appear and their operations (Section III-A), the object-wise dynamic program that produces all MCPs of the window (Section III-B), and the pruning rules for the duration threshold d and the size threshold k (Section III-C). Finally, we combine them into a complete algorithm and describe its implementation (Section III-D). Since t is fixed throughout this section, we write $F(o)$, $F(X)$, and $cover(S)$ for $F_t(o)$, $F_t(X)$, and $cover_t(S)$.

A. Appearance Bitmaps and the Object-wise Scan

Let $o_1, \ldots, o_n$ be the objects that appear in W_t. Each object o records its appearance frame set $F(o)$ in a w-bit bitmap, called its *appearance bitmap*: every frame of the window corresponds to one bit, and the j-th bit is 1 if and only if o appears in the j-th frame of the window. More generally, every frame set $S \subseteq W_t$ is represented by a w-bit bitmap in the same way, and we use a frame set and its bitmap interchangeably.

object	f_0	f_1	f_2	f_3	f_4	f_5
A	0	1	1	1	1	1
B	0	1	1	1	1	1
C	1	0	1	1	0	0
D	0	1	0	1	1	1

Fig. 2. The appearance bitmaps corresponding to the example in Fig. 1.

Two basic operations work on appearance bitmaps: the bitwise AND, which computes $S \cap S'$; and the population count, which counts the 1-bits of a bitmap and thus computes $|S|$. Both operate on machine words; with 64-bit words, a bitmap occupies $\lceil w/64 \rceil$ words, so each operation takes $\lceil w/64 \rceil$ word operations. Since $F(X) = \bigcap_{o \in X} F(o)$ by definition, ANDing the appearance bitmaps of the objects in an object set X yields the bitmap of its co-occurrence frame set $F(X)$, and a population count of the result yields $|F(X)|$, the number of frames in which X co-occurs.

Fig. 2 shows the appearance bitmaps for the example in Fig. 1. With $w = 5$ and $t = 4$, $W_4 = \{0, 1, 2, 3, 4\}$. Each bitmap lists frames 0–4 from left to right, and fits in a single machine word. Object A appears in frames 1–4, so $F(A) = \texttt{01111}$; likewise, $F(B) = \texttt{01111}$, $F(C) = \texttt{10110}$ (frames 0, 2, and 3), and $F(D) = \texttt{01011}$ (frames 1, 3, and 4). The co-occurrence frame set of the object set $\{A, B, D\}$ takes two bitwise ANDs, $F(\{A, B, D\}) = F(A) \wedge F(B) \wedge F(D) = \texttt{01111} \wedge \texttt{01111} \wedge \texttt{01011} = \texttt{01011}$, i.e., frames $\{1, 3, 4\}$, and a population count gives $|F(\{A, B, D\})| = 3$: $\{A, B, D\}$ co-occur in three frames.

We compute MCPs by an object-wise scanning: objects are added one at a time; and adding an object o to an object set X costs one bitwise AND, since $F(X \cup \{o\}) = F(X) \cap F(o)$. Two simple properties make this scan order attractive.

Lemma 1 (Monotonicity)**.** *For any object sets $\emptyset \neq X \subseteq Y$, we have $F(Y) \subseteq F(X)$; that is, adding objects to an object set never adds frames to its co-occurrence frame set.*

Lemma 1 follows directly from set intersection: ANDing with $F(o)$ can turn a 1 into a 0 but never a 0 into a 1. Hence, frame sets never grow during the object-wise scan, which makes pruning by d possible (Section III-C). Two questions remain: (i) does every frame set produced by intersecting objects correspond to an MCP of the window; and (ii) which object set should be kept when several object sets yield the same frame set? Lemma 2 answers both.

Lemma 2 (Frame sets of MCPs)**.** *For every non-empty object set Y with $F(Y) \neq \emptyset$, the pair $(cover(F(Y)), F(Y))$ is an MCP. Moreover, if some object set has exactly S as its co-occurrence frame set, then $cover(S)$ is the largest such set.*

Proof. Let $S = F(Y)$ and $X = cover(S)$. Since every object of Y appears in all frames of S, we have $Y \subseteq X$ and hence $F(X) \subseteq F(Y) = S$ by Lemma 1; since every object of X appears in all frames of S, we also have $S \subseteq F(X)$. Thus $S = F(X)$ and $X = cover(S)$, which are the two conditions of Definition 1. For the second claim, every object set Y' with

$F(Y') = S$ satisfies $Y' \subseteq cover(S)$, and $F(cover(S)) = S$ by the first claim applied to Y'. □

The first claim of Lemma 2 answers question (i): every non-empty frame set $F(Y)$ produced by the object-wise scan is the frame set of some MCP, and the scan never produces a frame set that corresponds to no MCP. In other words, the object-wise scan satisfies condition 1 of Definition 1 by construction. The second claim answers question (ii): for each frame set, it suffices to keep the largest object set that produces it. Once all objects are scanned, this set is $cover(S)$, so object-maximality is obtained without computing $cover(S)$ separately.

For example, in W_4 (Fig. 2), adding D and then C to $\{A\}$ shrinks its co-occurrence frame set from $\{1,2,3,4\}$ to $\{1,3,4\}$ and then to $\{3\}$, as Lemma 1 states. For $Y = \{A, D\}$, $F(Y) = \{1,3,4\}$. The objects that appear in all of frames 1, 3, and 4 are A, B, and D, so $cover(\{1,3,4\}) = \{A,B,D\}$. This yields the MCP $(\{A,B,D\}, \{1,3,4\})$, even though $(\{A,D\}, \{1,3,4\})$ itself is not an MCP. Among $\{D\}$, $\{A,D\}$, $\{B,D\}$, and $\{A,B,D\}$, which all have exactly $\{1,3,4\}$ as their co-occurrence frame set, it suffices to keep the largest, $\{A,B,D\}$.

B. Object-wise MCP Computation by Dynamic Programming

We process the objects one at a time and maintain a table $\mathcal{D}$. Let $\mathcal{D}_i$ denote the table after the first i objects have been processed, starting from the empty table $\mathcal{D}_0$. Each entry of $\mathcal{D}_i$ has the form $S \mapsto X$, also write as the pair (X, S). The key $S \subseteq W_t$ is a co-occurrence frame set, stored as a w-bit bitmap. The value $X = \mathcal{D}_i[S]$ is the largest subset of $\{o_1, \ldots, o_i\}$, whose co-occurrence frame set is exactly S; $\mathcal{D}_i[S]$ is undefined if no such subset exists. Since $F(Y \cup Z) = F(Y) \cap F(Z)$, the union of all subsets with frame set S also has frame set S, so this largest subset is unique: it consists of the objects among $\{o_1, \ldots, o_i\}$ that appear in every frame of S. Candidates whose frame set becomes empty are discarded. Processing an object may add new keys or replace the value of an existing key with a larger object set; without pruning, no key is ever removed.

When object $o = o_{i+1}$ is processed, every subset of $\{o_1, \ldots, o_{i+1}\}$ falls into one of the three cases: it excludes o, and is thus already accounted for in $\mathcal{D}_i$; it extends some subset X of $\{o_1, \ldots, o_i\}$ with o, which changes its frame set from $S' = F(X)$ to $S' \cap F(o)$; or it is the singleton $\{o\}$, with frame set $F(o)$. Let $P_i(S) = \{S' \in \mathrm{dom}(\mathcal{D}_i) \mid S' \cap F(o) = S\}$ be the keys of $\mathcal{D}_i$ that yield S when intersected with $F(o)$, where $\mathrm{dom}(\mathcal{D}_i)$ is the set of keys of $\mathcal{D}_i$. For every non-empty S, the Dynamic Programming (DP) recurrence is

$$\mathcal{D}_{i+1}[S] = \max\big(\, \mathcal{D}_i[S];\ \mathcal{D}_i[S'] \cup \{o\} \text{ for } S' \in P_i(S); \text{ and } \{o\} \text{ if } S = F(o) \,\big) \tag{1}$$

where $\max$ returns the candidate object set of largest cardinality, an undefined $\mathcal{D}_i[S]$ is not a candidate, and $\mathcal{D}_{i+1}[S]$ is undefined if there is no candidate.

The three kinds of candidates in Eq. (1) correspond to the three cases above: the first keeps the entries of $\mathcal{D}_i$, the second extends a recorded object set with o, and the third is the singleton $\{o\}$. In practice, Eq. (1) is evaluated by a single pass over $\mathcal{D}_i$ rather than by searching $P_i(S)$ for each S: each key S' is intersected with $F(o)$, and the resulting key S, if non-empty, is updated with $\mathcal{D}_i[S'] \cup \{o\}$ whenever that set is larger than the current value.

o_i	$\mathcal{D}_i$	$\mathcal{D}_i$ Without pruning	$\mathcal{D}_i$ With pruning ($d = k = 3$)
A	$\mathcal{D}_1$:	`01111` $\mapsto \{A\}$	`01111` $\mapsto \{A\}$
B	$\mathcal{D}_2$:	`01111` $\mapsto \{A,B\}$	`01111` $\mapsto \{A,B\}$
C	$\mathcal{D}_3$:	`01111` $\mapsto \{A,B\}$, `10110` $\mapsto \{C\}$, `00110` $\mapsto \{A,B,C\}$	`01111` $\mapsto \{A,B\}$, `10110` $\mapsto \{C\}$; `00110` pruned by Rule 1
D	$\mathcal{D}_4$:	`01111` $\mapsto \{A,B\}$, `10110` $\mapsto \{C\}$, `00110` $\mapsto \{A,B,C\}$, `01011` $\mapsto \{A,B,D\}$, `00010` $\mapsto \{A,B,C,D\}$	`01111` $\mapsto \{A,B\}$, `01011` $\mapsto \{A,B,D\}$; `10110` pruned by Rule 2

Fig. 3. The table $\mathcal{D}_i$ after each object of W_4 is processed in the order A, B, C, D, without pruning and with pruning for $d = k = 3$ (Algorithm 1).

Fig. 3 lists the table $\mathcal{D}_i$ that Eq. (1) yields after each object of W_4, processed in the order A, B, C, D without pruning. A yields `01111` $\mapsto \{A\}$, which B extends to $\{A,B\}$ on the same key; C adds `10110` $\mapsto \{C\}$ and extends $\{A,B\}$ to `00110` $\mapsto \{A,B,C\}$. D intersects the three keys of $\mathcal{D}_3$ with `01011`, producing keys `01011` and `00010`. On key `01011`, the candidates are the singleton $\{D\}$ and $\{A,B\} \cup \{D\}$ from key `01111`, so $\{A,B,D\}$ wins. On key `00010`, the candidates are $\{C,D\}$ from key `10110` and $\{A,B,C,D\}$ from key `00110`, so $\{A,B,C,D\}$ wins. The resulting table $\mathcal{D}_4$ holds five entries, which are exactly the five MCPs of W_4. Theorem 1 shows that this holds in general.

Theorem 1. *After all n objects of the window W_t have been processed, $\mathcal{D}_n[S]$ is defined if and only if S is the frame set of an MCP of W_t, and in that case $(\mathcal{D}_n[S], S)$ is that MCP. Hence $\mathcal{D}_n$ contains exactly $\mathrm{MCP}(W_t)$, the set of all MCPs in W_t.*

Proof. We show by induction on i that $\mathcal{D}_i[S]$ is the largest subset of the first i objects whose frame set is exactly S. A largest such subset of the first $i+1$ objects either excludes o_{i+1}, or consists of o_{i+1} and a possibly empty set X' of the first i objects with $F(X') \cap F(o_{i+1}) = S$. Replacing a non-empty X' by $\mathcal{D}_i[F(X')]$ does not decrease its size, so Equation (1) attains it. Conversely, every term of Equation (1) is the size of a subset whose frame set is exactly S: $\mathcal{D}_i[S]$, $\mathcal{D}_i[S'] \cup \{o_{i+1}\}$ for $S' \in P_i(S)$, and $\{o_{i+1}\}$ if $S = F(o_{i+1})$. Hence $\mathcal{D}_n[S]$ is the largest object set whose frame set is exactly S, which is $cover(S)$ by Lemma 2. Every key S of $\mathcal{D}_n$ is of the form $F(Y)$ with $S \neq \emptyset$, so $(\mathcal{D}_n[S], S)$ is an MCP by Lemma 2; conversely, the frame set $S = F(X)$ of every MCP (X, S) is generated by the object set X itself. □

C. Pruning by d and k

By Theorem 1, the unpruned table $\mathcal{D}$ holds every MCP of the window, most of which are not answers. Two rules discard candidates as early as possible.

a) Rule 1 (duration): Objects with $|F(o)| < d$ are excluded from the scan; the remaining objects, in processing order, form a sequence L of length m. During the scan, a

candidate whose frame set has fewer than d frames is discarded as soon as it is generated.

b) Rule 2 (size): Before the i-th object of L is processed, every entry (X, S) with $|X|+(m-i+1) < k$ is removed, since X cannot reach k objects even if all $m - i + 1$ unprocessed objects join it.

Adding the two rules to the DP yields the *pruned DP*: only the objects in L are processed by Eq. (1), one at a time; Rule 2 is applied before each object is processed, and every candidate $\mathcal{D}_i[S'] \cup \{o\}$ with fewer than d frames is discarded by Rule 1. The singleton object set $\{o\}$ needs no check: its frame set is $F(o)$, and Rule 1 admits o into L only if $|F(o)| \geq d$.

While Rule 1 is safe by Lemma 1, Rule 2 has two side effects: a removed entry is never extended by later objects, and its key may later be regenerated from another entry with a smaller object set. Lemma 3 shows that neither effect loses an answer.

Lemma 3. *Rules 1 and 2 never discard a candidate that leads to an answer: for every $(X, S) \in Ans_t$, once the pruned DP has processed all objects in L, the object set stored under key S is X.*

Proof. Let $(X, S) \in Ans_t$, so $|S| \geq d$ and $|X| \geq k$. Every object of X has $|F(o)| \geq |S| \geq d$ and thus belongs to L. For $i = 0, \ldots, m$, let $X_i = X \cap \{L[1], \ldots, L[i]\}$. We show by induction on i that, once the i-th object has been processed and $X_i \neq \emptyset$, $\mathcal{D}$ holds under key $F(X_i)$ an object set of at least $|X_i|$ objects. Consider $o = L[i+1]$. If $X_i \neq \emptyset$, the objects of $X \setminus X_i$ are among the $m - i$ unprocessed ones, so the object set stored under $F(X_i)$ has at least $|X_i| \geq |X| - (m - i) \geq k - (m - i)$ objects and survives Rule 2. If $o \notin X$, then $X_{i+1} = X_i$, and by the first term of Equation (1) this object set stays under key $F(X_i)$, possibly replaced by a larger one. If $o \in X$, then $F(X_{i+1}) = F(X_i) \cap F(o) \supseteq S$ by Lemma 1, so $|F(X_{i+1})| \geq d$; by the second term of Equation (1) (the third if $X_i = \emptyset$), key $F(X_{i+1})$ receives an object set of at least $|X_{i+1}|$ objects, which survives Rule 1. For $i = m$ we have $X_m = X$, so $\mathcal{D}[S]$ holds an object set of at least $|X|$ objects whose frame set is S; by Lemma 2, no such object set is larger than $cover(S) = X$, so $\mathcal{D}[S] = X$. □

Corollary 1. *Once the pruned DP has processed all objects in L, the entries (X, S) of $\mathcal{D}$ with $|X| \geq k$ form exactly Ans_t.*

Proof. By Lemma 3, every answer is in $\mathcal{D}$. Conversely, every entry (X, S) of $\mathcal{D}$ satisfies $F(X) = S$, and $|S| \geq d$ by Rule 1. If $|X| \geq k$, then $(cover(S), S)$ is an MCP by Lemma 2. Since $|cover(S)| \geq |X| \geq k$, this MCP is an answer, and Lemma 3 gives $X = cover(S)$. □

D. Algorithm and Implementation

Algorithm 1 implements the pruned DP. It first applies Rule 1 to generate L with m objects (line 1). The table $\mathcal{D}$ is a hash table keyed by bitmaps (line 2), and the hash value of each key is computed once over its $\lceil w/64 \rceil$ words. To process o, the algorithm first applies Rule 2 (line 5), takes a snapshot of the current entries (line 6), and inserts the singleton object set $\{o\}$ under key $F(o)$ (line 7, the third candidate of Eq. (1)). Then, for every entry (X', S') in the snapshot, it computes $S' \cap F(o)$ by a bitwise AND and, if the result has at least d frames, inserts $X' \cup \{o\}$ under that key (lines 8–11, the second candidate of Eq. (1)). Procedure INSERT keeps the larger of the new object set and the one already stored under the same key, which realizes the $\max$ of Eq. (1). Entries that are not replaced keep their values, which corresponds to the first candidate of Eq. (1). The snapshot ensures that only the entries of $\mathcal{D}_i$ are extended, as in Eq. (1), and that entries created while processing o are not extended by o again. Hence, after the i-th iteration, $\mathcal{D}$ equals the pruned DP table after the first i objects of L, and by Corollary 1, Algorithm 1 returns exactly Ans_t.

Algorithm 1: Object-wise computation of MCPs for a single window

```
   Input: appearance bitmaps F(o) of the objects in W_t; thresholds d
          and k
   Output: Ans_t
1  L ← {o ∈ W_t | |F(o)| ≥ d};  m ← |L|                    // Rule 1
2  D ← empty hash table keyed by bitmaps
3  for i ← 1 to m do
4      o ← L[i];  r ← m − i + 1                            // # unprocessed
5      D ← { (X, S) ∈ D | |X| + r ≥ k }                    // Rule 2
6      Q ← snapshot of the entries of D
7      INSERT(D, {o}, F(o))
8      foreach (X', S') ∈ Q do
9          S ← S' ∧ F(o)                                   // bitwise AND
10         if |S| ≥ d then                                 // Rule 1
11             INSERT(D, X' ∪ {o}, S)
12 return { (X, S) | (X, S) ∈ D, |X| ≥ k }
13 Procedure INSERT(D, X, S)
14     if D[S] is undefined or |D[S]| < |X| then
15         D[S] ← X                                        // keep the larger object set
```

The third column of Fig. 3 shows Algorithm 1 on W_4 with $d = k = 3$. All four objects appear in at least three frames, so $L = \langle A, B, C, D \rangle$ and $m = 4$. When C is processed, extending $\{A, B\}$ yields key `00110` with only two frames, discarded by Rule 1. Before D is processed, the entry `10110` $\mapsto \{C\}$ cannot reach three objects and is pruned by Rule 2. D then yields `01011` $\mapsto \{A, B, D\}$, and the algorithm returns $Ans_4 = \{(\{A, B, D\}, \{1, 3, 4\})\}$. Of the five entries in the unpruned table, only two remain, and only one of them is an answer.

IV. Incremental Maintenance

Consecutive windows share $w - 1$ frames, so their answer sets largely overlap, and recomputing each window from scratch repeats most of the work. This section therefore maintains Ans_t incrementally. Section IV-A introduces a circular encoding, together with the table $\mathcal{D}$ and the oldest-frame index built on it. Section IV-B characterizes how MCPs change when a frame expires or arrives. Section IV-C presents the per-frame maintenance, proves its correctness and the safety of permanent pruning, and traces it on the running example.

Section IV-D analyzes its cost. Again, dut to space constraints, please refer to our technical report [13] for all omitted proofs.

A. Circular Encoding for Incremental Maintenance

The bitmaps of Section III encode each frame by its position relative to the window. Each arrival therefore shifts every appearance bitmap by one bit and changes every key of $\mathcal{D}$, so nothing computed for W_{t-1} can be reused for W_t. We instead encode frames by their *absolute positions* modulo w.

Let $pos(i) = i \bmod w$ be the *position* of frame i; frame i is represented by bit $pos(i)$. The w frames of a window are consecutive, so they occupy w distinct positions; a frame keeps its position from its arrival until its expiration, and the new frame t takes over the position of the expiring frame $t - w$, since $pos(t) = pos(t-w)$. Every frame set S of the window is encoded in the same way: bit $pos(i)$ is 1 if and only if $i \in S$. Hence, a frame set that lies in both W_{t-1} and W_t has the same bitmap in both windows, and a slide changes only the bits of the expiring and the arriving frame. Within a single window, the circular encoding is the window-relative encoding rotated by the same offset for every bitmap. Rotation preserves bitwise ANDs and population counts, so the results of Section III and Algorithm 1 apply unchanged. Our incremental maintenance uses following three structures.

1) Appearance bitmaps. For each object o that appears in the current window W_t, we store the circularly encoded bitmap of its appearance frame set $F_t(o)$, together with its cardinality $|F_t(o)|$, which is updated with every bit change.

2) Table $\mathcal{D}$. As in Section III, $\mathcal{D}$ is a hash table keyed by bitmaps that maps the frame set S of each entry to its object set X. Unlike in Section III, $\mathcal{D}$ keeps only answers. After frame t has been processed, the entries of $\mathcal{D}$ are exactly the answers in Ans_t, so reporting the answers amounts to listing $\mathcal{D}$. Every entry is therefore an MCP of W_t. The only exception is transient: while a frame is being processed, an entry may temporarily fail to be an MCP, e.g., before it is merged or replaced (Section IV-C).

3) Oldest-frame index. The *oldest-frame index* I stores, for each position p, the list $I[p]$ of the keys of $\mathcal{D}$ whose oldest frame is at position p. Each key of $\mathcal{D}$ is registered in I in exactly one list, under its oldest frame. It is registered when it is inserted into $\mathcal{D}$ and registered again whenever its frame set changes. The expiring frame $t - w$ is the oldest frame of W_{t-1}, so a frame set of W_{t-1} contains $t - w$ if and only if its oldest frame is $t - w$. Hence $I[pos(t)]$ lists exactly the entries affected by the expiration, and no other entry needs to be inspected. The oldest frame of a key is the first 1-bit found when its bitmap is scanned circularly, starting from the position of the window's oldest frame. With a trailing-zero count instruction, this scan takes $\lceil w/64 \rceil$ word operations.

Incremental maintenance processes each new frame t in two steps. 1) Expiration. If $t \geq w$, frame $t - w$ expires. Its bit is cleared from the bitmaps of its objects O_{t-w}, bitmaps that become empty are removed, and the entries in $I[pos(t-w)]$ are trimmed. 2) Arrival. Bit $pos(t)$ is set in the bitmaps of the objects in O_t, and the entries in $\mathcal{D}$ that contain frame t are computed by Algorithm 1 over the objects of O_t only. All other entries stay untouched.

	position p	0	1	2	3	4
$t=4$	frame	0	1	2	3	4
	$F_4(A)$	0	1	1	1	1
	$F_4(B)$	0	1	1	1	1
	$F_4(C)$	1	0	1	1	0
	$F_4(D)$	0	1	0	1	1
$t=5$	frame	**5**	1	2	3	4
	$F_5(A)$	**1**	1	1	1	1
	$F_5(B)$	**1**	1	1	1	1
	$F_5(C)$	**0**	0	1	1	0
	$F_5(D)$	**1**	1	0	1	1

(a) Appearance bitmaps

X	$\{C\}$	$\{A,B\}$	$\{A,B,D\}$	$\{A,B,C\}$
$t=4$	10110	01111	01011	00110
$t=5$	–	11111	11011	00110

(b) Table $\mathcal{D}$

p	$I[p]$, $t=4$	$I[p]$, $t=5$
0	10110	–
1	01111, 01011	11111, 11011
2	00110	00110
3, 4	–	–

(c) Oldest-frame index I

Fig. 4. Circular encoding of the running example with $w = 5$. (a) Appearance bitmaps at $t = 4$ and $t = 5$. (b) The table $\mathcal{D}$ and (c) the oldest-frame index I for $CQ(5, 2, 1)$ at $t = 4$ and $t = 5$; each column of (b) is an entry $S \mapsto X$.

For example, Fig. 4(a) shows the circular encoding of the running example with $w = 5$. When frame 5 arrives, it takes over position 0 from the expired frame 0, while frames 1–4 keep positions 1–4. Consequently, only bit at position 0 changes: it becomes 1 for A, B, and D, and 0 for C, as $O_5 = \{A, B, D\}$. The keys of $\mathcal{D}$ are encoded in the same way, so a frame set that contains neither the expired frame 0 nor the arrived frame 5 keeps its key. For example, the entry $\texttt{00110} \mapsto \{A, B, C\}$ in Fig. 4(b) has its bit 0 ($p(0) = p(5) = 0$) being 0 at both $t = 4$ and $t = 5$. Its key is therefore unaffected by the slide, and the entry stays valid. Under the window-relative encoding, by contrast, the meaning of every bitmap would shift instead, e.g., $F_5(D)$ with circular-encoding bitmap `11011` would be represented as `10111`, i.e., object D appears in frames $\{1, 3, 4, 5\}$ at $t = 5$ and so would every key of $\mathcal{D}$.

In Fig. 4(c), when frame 0 expires, only the entry in $I[0]$ is trimmed. At $t = 5$, bit 0 stands for frame 5, so the oldest frame of a key is found by scanning from $pos(1) = 1$, the position of the oldest frame of W_5: `11111` and `11011` meet their first 1 at bit 1 and are registered in $I[1]$, exactly the entries to trim when frame 1 expires next.

B. Properties of MCPs under Window Sliding

This subsection answers the central question of incremental maintenance: given the answer set of W_{t-1}, sotred in $\mathcal{D}$, how do we obtain the answer set of W_t? Each slide consists of an expiration and an arrival, so we characterize how the set of MCPs changes in each step. These characterizations serve two purposes. First, they tell which entries each step must process, and how. Second, they show that the processing needs only the stored answers and the appearance bitmaps, never a candidate that was pruned, which estabslishes that permanent pruning is safe. Lemma 4 lays the groundwork, Lemma 5 and Lemma 6 characterize the expiration and the arrival, respectively.

Generalized notation. When frame t arrives, the window changes from W_{t-1} to W_t in two steps. If $t \geq w$, the oldest frame $e = t - w$ is removed, giving $W^\circ = W_{t-1} \setminus \{e\}$. The new frame t is then added, giving $W_t = W^\circ \cup \{t\}$; if $t < w$, then $W^\circ = W_{t-1}$. The intermediate frame set W° is not a window, so to reason about it we extend the notation to an

arbitrary frame set W. The *appearance frame set* of an object o in W is $F_W(o) = \{i \in W \mid o \in O_i\}$, i.e., the frames of W in which o appears. The *co-occurrence frame set* of a non-empty object set X in W is $F_W(X) = \bigcap_{o \in X} F_W(o)$, i.e., the frames of W in which all objects in X co-appear. For a frame set $S \subseteq W$, $cover_W(S) = \{\, o \mid S \subseteq F_W(o) \,\}$ is the set of objects that appear in every frame of S. $\mathrm{MCP}(W)$ denotes the set of MCPs of W, defined by Definition 1 with W in place of W_t, and $Ans(W) = \{\, (X, S) \in \mathrm{MCP}(W) \mid |S| \geq d,\ |X| \geq k \,\}$ is the answer set over W. Thus $F_t = F_{W_t}$, $cover_t = cover_{W_t}$, and $Ans_t = Ans(W_t)$. Lemmas 1 and 2 hold for any W, with the same proofs.

Window-independent cover. We first show that the object set of an entry is not affected by change to other frames of the window. Otherwise, even an entry that contains neither the expiring nor the new frame might need updating, and processing only the affected entries would be impossible.

Lemma 4 (Window-independent cover)**.** *For every non-empty frame set $S \subseteq W$, $cover_W(S) = \bigcap_{i \in S} O_i$.*

Proof. An object o belongs to $cover_W(S)$ if and only if $S \subseteq F_W(o)$, i.e., $o \in O_i$ for every $i \in S$. □

Hence, $cover_W(S)$ depends only on S, not on W, as long as $S \subseteq W$, and we write it as $cover(S)$. It follows that a slide can change an entry in only two ways: by removing the expiring frame from its frame set, or by adding the new frame to it.

Expiration. Removing frame e takes one frame away from every MCP that contains e, while MCPs that do not contain e keep their frame sets. A trimmed MCP may end up with the same frame set of an MCP that does not contain e. This raises two questions. (i) Can every MCP of W° be obtained from an MCP of W, or can new MCPs unrelated to those of W appear? (ii) When several MCPs end up with the same frame set, which object set should be kept? The two claims of Lemma 5 answer these questions in turn.

Lemma 5 (Expiration)**.** *Let W be a set of frames, $e \in W$, and $W^\circ = W \setminus \{e\}$.*

1) *For every MCP (X', S') of W°, the pair $(X', F_W(X'))$ is an MCP of W, and $F_W(X') \setminus \{e\} = S'$.*
2) *For every MCP (X, S) of W with $S \setminus \{e\} \neq \emptyset$, the frame set $S \setminus \{e\}$ is the frame set of an MCP of W°, whose object set $cover(S \setminus \{e\})$ is the largest object set among the MCPs of W whose frame sets become $S \setminus \{e\}$ when e is removed.*

Proof. (1) Let $S = F_W(X')$. Since $F_{W^\circ}(X') = F_W(X') \setminus \{e\}$ and $F_{W^\circ}(X') = S'$ by condition 1 of Definition 1, we have $S \setminus \{e\} = S'$. Every object of X' appears in all frames of S, so $X' \subseteq cover(S)$; conversely, $S' \subseteq S$ gives $cover(S) \subseteq cover(S') = X'$. Hence $cover(S) = X'$, and (X', S) satisfies both conditions of Definition 1 for W.

(2) Let $S' = S \setminus \{e\}$ and $X' = cover(S')$, so $X' \supseteq cover(S) = X$. By Lemma 1, $F_{W^\circ}(X') \subseteq F_{W^\circ}(X) = F_W(X) \setminus \{e\} = S'$, and $F_{W^\circ}(X') \supseteq S'$ by the definition of *cover*; hence (X', S') is an MCP of W°. Every MCP (X'', S'') of W with $S'' \setminus \{e\} = S'$ satisfies $X'' = cover(S'') \subseteq cover(S') = X'$, and by (1), $(X', F_W(X'))$ is itself such an MCP. □

In words, the MCPs of W° are obtained from those of W by removing e from every frame set and keeping the largest object set for each resulting frame set. By claim (1), every answer of W° comes from an answer of W that has the same object set and at least as many frames. The expiration therefore only trims stored entries and never needs a pruned candidate. This is the key to the safety of permanent pruning.

Arrival. The new frame t affects MCPs in three ways: i) it creates new MCPs whose frame sets contain t; ii) it extends by t each MCP whose objects all appear in t, so that the old pair is no longer an MCP; and iii) it leaves all other MCPs unchanged. The three claims of Lemma 6 characterize these cases.

Lemma 6 (Arrival)**.** *Let W° be a set of frames, $t \notin W^\circ$ a new frame, and $W = W^\circ \cup \{t\}$.*

1) *The MCPs of W whose frame sets contain t are exactly the pairs $(cover(F_W(Y)), F_W(Y))$ for non-empty $Y \subseteq O_t$, and their object sets lie in O_t. Consequently, Algorithm 1 applied to the bitmaps $F_W(o)$ of the objects $o \in O_t$ returns exactly the answers in $Ans(W)$ whose frame sets contain t.*
2) *Let (X, S) be an MCP of W°. If $X \not\subseteq O_t$, then (X, S) is an MCP of W; otherwise, $(X, S \cup \{t\})$ is an MCP of W and (X, S) is not.*
3) *Every MCP of W whose frame set does not contain t is an MCP of W°.*

Proof. (1) If (X, S) is an MCP of W with $t \in S$, then $X = cover(S) \subseteq O_t$ by Lemma 4, and $S = F_W(X)$. Conversely, every non-empty $Y \subseteq O_t$ has $t \in F_W(Y)$, and $(cover(F_W(Y)), F_W(Y))$ is an MCP of W by Lemma 2. The proofs of Theorem 1, Lemma 3, and Corollary 1 therefore carry over when the scan is restricted to the objects of O_t: every frame set produced contains t, and the object set of every answer that contains t lies in O_t.

(2) If $X \not\subseteq O_t$, then $F_W(X) = F_{W^\circ}(X) = S$, and $cover(S) = X$ by Lemma 4. If $X \subseteq O_t$, then $F_W(X) = S \cup \{t\} \neq S$, so (X, S) violates condition 1 of Definition 1, whereas $cover(S \cup \{t\}) = cover(S) \cap O_t = X$.

(3) Let (X, S) be an MCP of W with $t \notin S$. Then $F_{W^\circ}(X) = F_W(X) \setminus \{t\} = S$ and $cover(S) = X$. □

Claim (1) justifies running Algorithm 1 only on the objects of the new frame. The MCPs that contain t are recomputed from their appearance bitmaps rather than derived from stored entries, so this step needs no pruned candidate either. Claim (2) gives the exact condition under which a stored entry becomes invalid, i.e., the replacement test of Section IV-C. Claim (3) shows that no other entry needs to be checked.

Algorithm 2: Per-frame maintenance

Input: new frame t with objects O_t; window size w; thresholds d and k; bitmaps $F_{t-1}(o)$, table $\mathcal{D} = Ans_{t-1}$, and index I
Output: bitmaps $F_t(o)$, table $\mathcal{D} = Ans_t$, and index I

1 $p \leftarrow t \bmod w$
2 **if** $t \geq w$ **then** // expiration
3 **foreach** $o \in O_{t-w}$ **do**
4 $F_{W^\circ}(o) \leftarrow F_{t-1}(o)$ with bit p cleared
5 **if** $F_{W^\circ}(o)$ is empty **then** remove it // remove
6 **foreach** $S \in I[p]$ **do**
7 $X \leftarrow \mathcal{D}[S]$; remove S from $\mathcal{D}$
8 $S' \leftarrow S$ with bit p cleared
9 **if** $|S'| \geq d$ **then** INSERT$(\mathcal{D}, X, S')$ // Rule 1
10 $I[p] \leftarrow \emptyset$
11 **foreach** $o \in O_t$ **do** // arrival
12 $F_t(o) \leftarrow F_{W^\circ}(o)$ with bit p set
13 $\mathcal{D}' \leftarrow$ Algorithm 1 on $\{F_t(o) \mid o \in O_t\}$ // Arrival DP
14 **foreach** $(X, S) \in \mathcal{D}'$ **do**
15 $S^- \leftarrow S$ with bit p cleared
16 **if** $\mathcal{D}[S]$ is undefined and $|\mathcal{D}[S^-]| = |X|$ **then** // replaced
17 remove S^- from $\mathcal{D}$ and I
18 INSERT$(\mathcal{D}, X, S)$

19 **Procedure** INSERT$(\mathcal{D}, X, S)$
20 **if** $\mathcal{D}[S]$ is undefined **then**
21 $\mathcal{D}[S] \leftarrow X$; add S to $I[pos(\text{oldest frame of } S)]$
22 **else if** $|\mathcal{D}[S]| < |X|$ **then** $\mathcal{D}[S] \leftarrow X$

C. Per-frame Maintenance

Algorithm 2 updates $\mathcal{D}$ from Ans_{t-1} to Ans_t for every new frame $t > t_0$, where t_0 is the time when the query starts and $\mathcal{D}$ is initialized at t_0 by Algorithm 1. Let $p = pos(t)$. The new frame t and the expiring frame $t - w$ share this position.

Expiration. If $t \geq w$, frame $e = t - w$ leaves the window (line 2). For each object $o \in O_e$, the algorithm clears bit p of its bitmap $F_{t-1}(o)$, which yields $F_{W^\circ}(o)$ (line 4). Every other object already has a 0 at this bit, so $F_{W^\circ}(o) = F_{t-1}(o)$. In our implementation, O_e is obtained from a *frame ring* of length w that stores the object set of each frame of the window: the object set of frame i occupies slot $pos(i)$, and O_t of the new frame t overwrites the slot left by the expiring frame $t - w$. An object whose $F_{W^\circ}(o)$ is empty no longer appears in the window, and its bitmap is recycled (line 5). Hence only the objects of the current window keep appearance bitmaps.

Next, the algorithm trims the entries that contain e, which are exactly those in $I[p]$ (lines 6–9). Each such entry (X, S) is removed from $\mathcal{D}$, and bit p is cleared from its key S. If $S' = S \setminus \{e\}$ still has at least d frames, X is put back into $\mathcal{D}$ under key S' with INSERT (Rule 1); otherwise, it is discarded. The procedure INSERT follows the procedure of Algorithm 1: if the key is already in $\mathcal{D}$, it keeps the larger object set. Its only additional step registers a new key in I under its oldest frame (line 21). This merges the entries whose frame sets coincide once e is removed, as required by Lemma 5. Entries that do not contain e are not touched in this step.

Arrival. Frame t then enters the window. Bit p is set in the bitmap $F_{W^\circ}(o)$ for every object $o \in O_t$, which yields $F_t(o)$. If o newly appears, $F_{W^\circ}(o)$ is empty and a new bitmap is created (lines 11- 12). For every object o' not in O_t, $F_t(o') = F_{W^\circ}(o')$. In our implementation, these updates are made in place on a single bitmap per object, and O_t is stored in slot p of the frame ring for the expiration of frame t. The entries that contain t are recomputed from the bitmaps rather than derived from stored entries. By Lemma 6(1), running Algorithm 1 with both pruning rules on the bitmaps $F_t(o)$ of the objects of O_t returns exactly the answers whose frame sets contain t (line 13). We call this run the *arrival DP*. It uses its own working table $\mathcal{D}'$, separate from the maintained table $\mathcal{D}$. Each answer $(X, S) \in \mathcal{D}'$ it returns is inserted into $\mathcal{D}$ with INSERT (line 18). After the expiration, no key of $\mathcal{D}$ contains bit p, so each insertion creates a new key.

Before each insertion, the *replacement test* checks the key $S^- = S \setminus \{t\}$ (line 16). The test fails if $\mathcal{D}$ has no entry under S^-. If $\mathcal{D}$ stores an object set under S^-, that set is $cover(S^-) \supseteq cover(S) = X$. It therefore has the same size as X if and only if it equals X, i.e., if and only if all of its objects appear in frame t. In that case, by Lemma 6(2), the old entry (X, S^-) is no longer an MCP, and it is replaced by (X, S). After the expiration, no key of $\mathcal{D}$ contains bit p, so INSERT always inserts a new key here. Entries that contain neither e nor t are not touched.

Correctness. Theorem 2 shows that Algorithm 2 maintains exactly the answer set after every frame.

Theorem 2. *For every $t \geq t_0$, after Algorithm 2 has processed frame t, the table $\mathcal{D}$ equals Ans_t.*

Proof. By induction on t. After initialization, $\mathcal{D} = Ans_{t_0}$ by Corollary 1; if the query starts with the stream, both the window and $\mathcal{D}$ are empty. Assume $\mathcal{D} = Ans(W_{t-1})$ before frame t. We first show that $\mathcal{D} = Ans(W^\circ)$ after the expiration, which is immediate if $t < w$. Let $(X', S') \in Ans(W^\circ)$. By Lemma 5(1), (X', S) with $S = F_{W_{t-1}}(X')$ is an MCP of W_{t-1} with $|S| \geq |S'| \geq d$ and $|X'| \geq k$, so $\mathcal{D}[S] = X'$ before the expiration. Every other entry of $\mathcal{D}$ whose frame set becomes S' after trimming has a proper subset of X' as its object set, since distinct MCPs have distinct object sets and X' is the largest by Lemma 5(2). Hence $\mathcal{D}[S'] = X'$ after the expiration. Conversely, let $\mathcal{D}[S'] = X$ after the expiration. This entry results from an answer (X, S) of W_{t-1} with $S \setminus \{e\} = S'$ and $|S'| \geq d$. By Lemma 5(2), $(cover(S'), S')$ is an MCP of W° with $|cover(S')| \geq |X| \geq k$, i.e., an answer of W°, so $X = cover(S')$ by the first part.

Next, by Lemma 6(1), line 13 returns exactly the answers of W_t that contain t, and all of them are inserted into $\mathcal{D}$. Every answer of W_t that does not contain t is an answer of W° with the same object and frame sets by Lemma 6(3), so it is already in $\mathcal{D}$; its object set is not a subset of O_t, so the replacement test does not remove it. Finally, let (X, S) be an answer of W° that is not an MCP of W_t, i.e., $\mathcal{D}[S] = X$. By Lemma 6(2), $X \subseteq O_t$ and $(X, S \cup \{t\})$ is an MCP of W_t; it is an answer that contains t, so it is returned in line 13, and the replacement test removes the key S. The object sets of the remaining entries are not subsets of O_t, so by Lemma 6(2), these entries are still MCPs of W_t with unchanged frame and

Step	Table $\mathcal{D}$
initialization at $t_0 = 4$	$\texttt{01111} \mapsto \{A, B\}$, $\texttt{01011} \mapsto \{A, B, D\}$, $\texttt{10110} \mapsto \{C\}$, $\texttt{00110} \mapsto \{A, B, C\}$
expire frame 0: $F_{W^\circ}(C) = \texttt{00110}$	$\texttt{01111} \mapsto \{A, B\}$, $\texttt{01011} \mapsto \{A, B, D\}$, $\texttt{00110} \mapsto \{A, B, C\}$
arrive frame 5: $F_5(A) = F_5(B) = \texttt{11111}$, $F_5(D) = \texttt{11011}$	$\texttt{11111} \mapsto \{A, B\}$, $\texttt{11011} \mapsto \{A, B, D\}$, $\texttt{00110} \mapsto \{A, B, C\}$

Fig. 5. Algorithm 2 on the running example for $CQ(5, 2, 1)$: initialization at $t_0 = 4$, followed by frame 5. At $t = 5$, position 0 holds frame 5.

object sets, and hence still answers. Therefore $\mathcal{D} = Ans(W_t)$ after the arrival. □

Theorem 2 implies that both pruning rules are permanent. $\mathcal{D}$ never holds an entry with fewer than d frames or fewer than k objects, nor any candidate discarded by the arrival DP, and Algorithm 2 never revisits them. This is safe because no answer of W_t is derived from a pruned candidate: By Lemma 5(1), every answer of W° comes from an answer of W_{t-1} with the same object set and at least as many frames, and every answer that contains t is recomputed from the bitmaps without consulting $\mathcal{D}$. A pruned candidate thus never needs to be stored; if its objects later co-occur in enough frames, the arrival DP finds them again from their bitmaps.

For example, we run Algorithm 2 on the running example with $w = 5$ and $CQ(5, 2, 1)$ (Fig. 5). Starting at $t_0 = 4$, Algorithm 1 initializes $\mathcal{D}$ to the four MCPs of W_4 with at least two frames. When frame 5 arrives, frame 0 at position $p = 0$ expires. Bit 0 is cleared only in $\texttt{10110}$, which stays non-empty and is not recycled. The only entry in $I[0]$, $\texttt{10110} \mapsto \{C\}$, is trimmed to $\texttt{00110}$ and merged into the stored entry $\texttt{00110} \mapsto \{A, B, C\}$; without frame 0, C appears only in frames 2 and 3, together with A and B. Bit 0 (representing frame 5) is set for A, B, and D (Fig. 4(a)), the arrival DP returns $\texttt{11111} \mapsto \{A, B\}$ and $\texttt{11011} \mapsto \{A, B, D\}$. These pass the replacement test again and replace $\texttt{01111} \mapsto \{A, B\}$ and $\texttt{01011} \mapsto \{A, B, D\}$. The entry $\texttt{00110} \mapsto \{A, B, C\}$ is untouched and remains an answer. These three entries form Ans_5. For $CQ(5, 3, 3)$, the initialization keeps only $\texttt{01011} \mapsto \{A, B, D\}$, the expiration touches nothing, and the arrival replaces it with $\texttt{11011} \mapsto \{A, B, D\}$.

D. Complexity Analysis

Time per frame. The expiration clears bit p in each of the $|O_{t-w}|$ bitmaps of the expiring frame. The bit lies in a single machine word, and and the maintained frame counts show in $O(1)$ whether a bitmap has become empty. It then trims the $C_e = |I[pos(t)]|$ entries that contain frame $t - w$, with $O(\lceil w/64 \rceil)$ word operations each. The arrival sets $|O_t|$ bits and runs the arrival DP on O_t, intersecting each object with every entry of the working table $\mathcal{D}'$. Let $|\mathcal{D}'|$ be the largest size of $\mathcal{D}'$ during the run. The arrival DP then takes $O(|O_t| \cdot |\mathcal{D}'| \cdot \lceil w/64 \rceil)$ word operations, which also covers the replacement tests and insertions of the at most $|\mathcal{D}'|$ answers returned. In total, frame t takes $O\big(|O_{t-w}| + |O_t| + (C_e + |O_t| \cdot |\mathcal{D}'|) \cdot \lceil w/64 \rceil\big)$ word operations, and entries that contain neither the expiring nor the new frame cost nothing. Unlike a rerun of Algorithm 1 on the whole window, the arrival DP scans only the objects of the new frame instead of all n_t objects of W_t, and its working table holds only candidates that contain frame t.

Worst case. Every key of the arrival DP is $F_t(Y)$ for some $Y \subseteq O_t$. Every entry that contains frame $t - w$ has its object set in O_{t-w}. Hence $|\mathcal{D}'| \leq 2^{|O_t|}$ and $C_e \leq 2^{|O_{t-w}|}$. The cost of a frame is therefore bounded by a function of the number of objects per frame, independent of the number of MCPs in the window and of the length of the stream. In practice, C_e and $|\mathcal{D}'|$ are far below these bounds and shrink as d grows. Rule 1 discards objects and candidates with fewer than d frames, and Rule 2 discards object sets that cannot reach k objects. By contrast, the frame-wise methods of [1] maintain all MCPs of the window regardless of d and visit every maintained state in each frame. Their cost per frame grows with the number of MCPs, which can exceed $|Ans_t|$ by orders of magnitude.

Space. OWIM keeps one appearance bitmap for each of the n_t objects in W_t, the $\sum_{i \in W_t} |O_i|$ object identifiers in the frame ring, and the $|Ans_t|$ entries of $\mathcal{D}$, each with a bitmap and an object set, and registered in I, in total $O\Big(n_t\lceil w/64 \rceil + \sum_{i \in W_t} |O_i| + \sum_{(X,S) \in Ans_t} \big(\lceil w/64 \rceil + |X|\big)\Big)$. Without recycling, a bitmap would be kept for every object seen since the start of the stream, and the space would grow without bound.

TABLE II
PARAMETER SETTINGS (DEFAULTS IN BOLD)

Param.	Meaning	Values
w	window size (frames)	**300**, 400, 500, 600, 900, 1200, 1800
d	duration threshold (frames)	1, 30, 60, 120, 180, 210, **240**, 270
k	size threshold (objects)	**1**, 2, 3, 5, 7, 9
p_o	max. reuses of an identifier	**0**, 1, 2, 3
p_r	fraction of removed objects	**0**, 0.1, 0.2, 0.3

V. EXPERIMENTS

All experiments run on an Intel Core i7-13700F CPU with 32 GB of memory under Windows 11. OWIM is implemented in Java 8, and the competitors use the unmodified Java 8 implementation of the authors of [1].

A. Experimental Setup

Datasets. For a direct comparison, we use the object streams of [1]. These are the detection and tracking results that the authors of [1] obtained with YOLOv3 [14] and DeepSORT [12]. They cover six regular and three long videos. V1 and V2 are two synthetic videos from the Visual Road benchmark [15], showing rainy weather with light traffic and post-rain conditions with heavy traffic, respectively. D1 and D2 (MVI_40171 and MVI_40751 from UA-DETRAC [16]) are captured by static cameras. M1 and M2 (MOT16-06 and MOT16-13 from MOT16 [17]) are captured by moving cameras. The three long videos are public YouTube videos [18]–[20], with about 35,000 to 53,000 frames each.

Competitors. We compare OWIM with the three state maintenance methods of [1], NAIVE, MFS, and SSG, all of which

Fig. 6. Total maintenance time

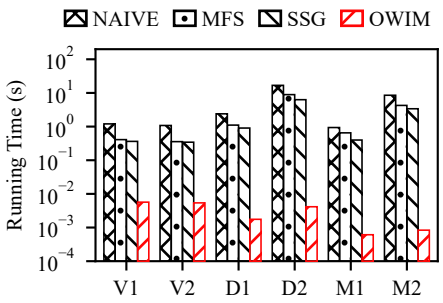


TABLE III
STATES (METHODS OF [1]) AND ENTRIES OF $\mathcal{D}$ (OWIM) MAINTAINED AND ACCESSED PER FRAME

	Methods of [1]		OWIM		
Dataset	NAIVE	MFS/SSG	maint.	acc.	Answers
V1	408.2	366.4	4.2	4.9	4.2
V2	325.9	289.5	3.8	5.5	3.8
D1	1348.2	1244.7	1.1	0.9	1.1
D2	6813.5	6623.2	2.3	3.2	2.3
M1	1770.8	1677.5	0.0	0.0	0.0
M2	7779.1	6882.0	0.7	0.6	0.7

TABLE IV
LONG VIDEOS: TOTAL TIME, AND STATES (SSG) OR ENTRIES (OWIM) PER FRAME

		L1	L2	L3
Frames		51010	53405	35376
Time (s)	NAIVE	879.3	194.5	184.1
	MFS	427.2	71.7	69.4
	SSG	308.2	61.4	54.2
	OWIM	0.132	0.216	0.271
SSG/OWIM		2342×	284×	200×
Per frame	SSG	8226.4	1742.8	2492.5
	OWIM	4.6	7.0	10.3

adopt Frame-Wise Incremental Maintenance. MFS and SSG with pruning by k (they cannot be pruned by d) are also provided and denoted by MFS_O and SSG_O. As this pruning has no effect when $k = 1$ and only adds the cost of evaluating the query on every new state, we use MFS and SSG when $k = 1$ and MFS_O and SSG_O when $k > 1$; NAIVE runs unchanged under all parameters.

Parameter settings. Table II lists the parameters, with the defaults underlined; each experiment varies one parameter and sets the others to their defaults, except the experiment on k. The defaults follow [1]: $w = 300$ and $d = 240$, i.e., objects that appear together for at least 8 of 10 seconds at 30 frames per second, and $k = 1$, since the state maintenance experiments of [1] do not constrain the size of object sets. The occlusion parameter p_o, the maximum number of times that the identifier of an object that has left the view is reused, simulating more occlusions, and the removal ratio p_r, the fraction of objects randomly removed from a video, varying the number of objects, also follow [1]; we use the files generated by its authors, which cover only the six regular videos. Beyond the values of [1], we extend d from 180–270 to 1–270, since d has the largest impact on the cost of OWIM. $d = 1$ shows the cost without pruning. We vary k from 1 to 9 to compare pruning by k with MFS_O and SSG_O, setting $d = 30$, since with $d = 240$ no dataset has an object set of five or more objects that co-occur in 240 frames. Finally, we extend w to 1800, i.e., 60 seconds, which makes each bitmap operation, taking $\lceil w/64 \rceil$ word operations, more expensive.

B. Overall Performance

Six regular videos. Fig. 6 shows the total maintenance time on the six regular videos under the default parameters ($w = 300$, $d = 240$, $k = 1$). OWIM takes only 0.6–5.7 ms, versus 0.34–6.30 s for SSG, 0.36–8.91 s for MFS, and 0.94–16.81 s for NAIVE; it is 64–4,055 times faster than SSG and 200–10,219 times faster than NAIVE. As in [1], SSG is the fastest baseline: it is 2.4–3.3 times faster than NAIVE and takes 4%–39% less time than MFS.

Table III explains the gap. For OWIM, the accessed entries are those trimmed at expiration plus the intersections performed by Algorithm 1. Since the methods of [1] do not prune by d, at every frame they maintain and access all MCPs of the window: 290–6,882 states for MFS and SSG, and 326–7,779 for NAIVE, which also keeps object sets that are not MCPs. Yet there are only 0–4.2 answers per frame on average; since its table is exactly the answer set, OWIM maintains only these entries and accesses only 0–5.5 entries per frame. The gap is thus determined by the ratio of the number of MCPs to the number of answers. It is the largest on D2 and M2, whose windows contain 6,623 and 6,882 MCPs on average, where OWIM is 1,518 and 4,055 times faster than SSG, and the smallest on V1 and V2, where objects stay long and rarely enter or leave the view, so the windows contain only 290–366 MCPs; even there, OWIM is 64 times faster. On M1, captured by a moving camera with the shortest object durations, no group of objects co-occurs in 240 frames, so the answer set and the table of OWIM stay empty: each frame only updates the bitmaps of the objects in the new and the expiring frames, and Rule 1 discards every object of the new frame, as each appears in fewer than 240 frames, while the methods of [1] still maintain 1,678 states per frame.

Long videos. On the three long videos of 35,000 to 53,000 frames, under the default parameters (Table IV), the methods of [1] take 54–879 s and OWIM only 0.13–0.27 s, i.e., 200–2,342 times faster than SSG and 680–6,682 times faster than NAIVE. The baselines again agree with [1]: SSG and MFS are 2.9–3.4 and 2.1–2.7 times faster than NAIVE, versus 3.2 and 2.6 times reported there for a long video. The gap is the largest on L1, which has about twice as many objects per frame as the other two: SSG maintains 8,226 states per frame and takes 308 s, whereas OWIM holds only 4.6 entries on average and its total time on L1 is the shortest of the three videos, so its cost depends on the number of answers rather than on the number of objects or MCPs. Since its table holds only the current answers and the bitmaps of objects that have left the window are recycled, the data maintained by OWIM does not grow with the length of the stream.

C. Effect of Query Parameters

Figs. 7(a)–(r) show the total maintenance time of all methods on the six videos as d, k, and w vary.

Varying d. In Figs. 7(a)–(f), the methods of [1] maintain all MCPs regardless of d, so on each dataset their longest time is only 1.1–1.5 times their shortest, whereas OWIM becomes 6–261 times faster from $d = 1$ to $d = 270$. To isolate the effect of permanent pruning, we add OWIM-NP, which maintains $\mathcal{D}$ with $d = k = 1$ and filters the answers by d and k only when reporting them; like the methods of [1], it maintains all MCPs

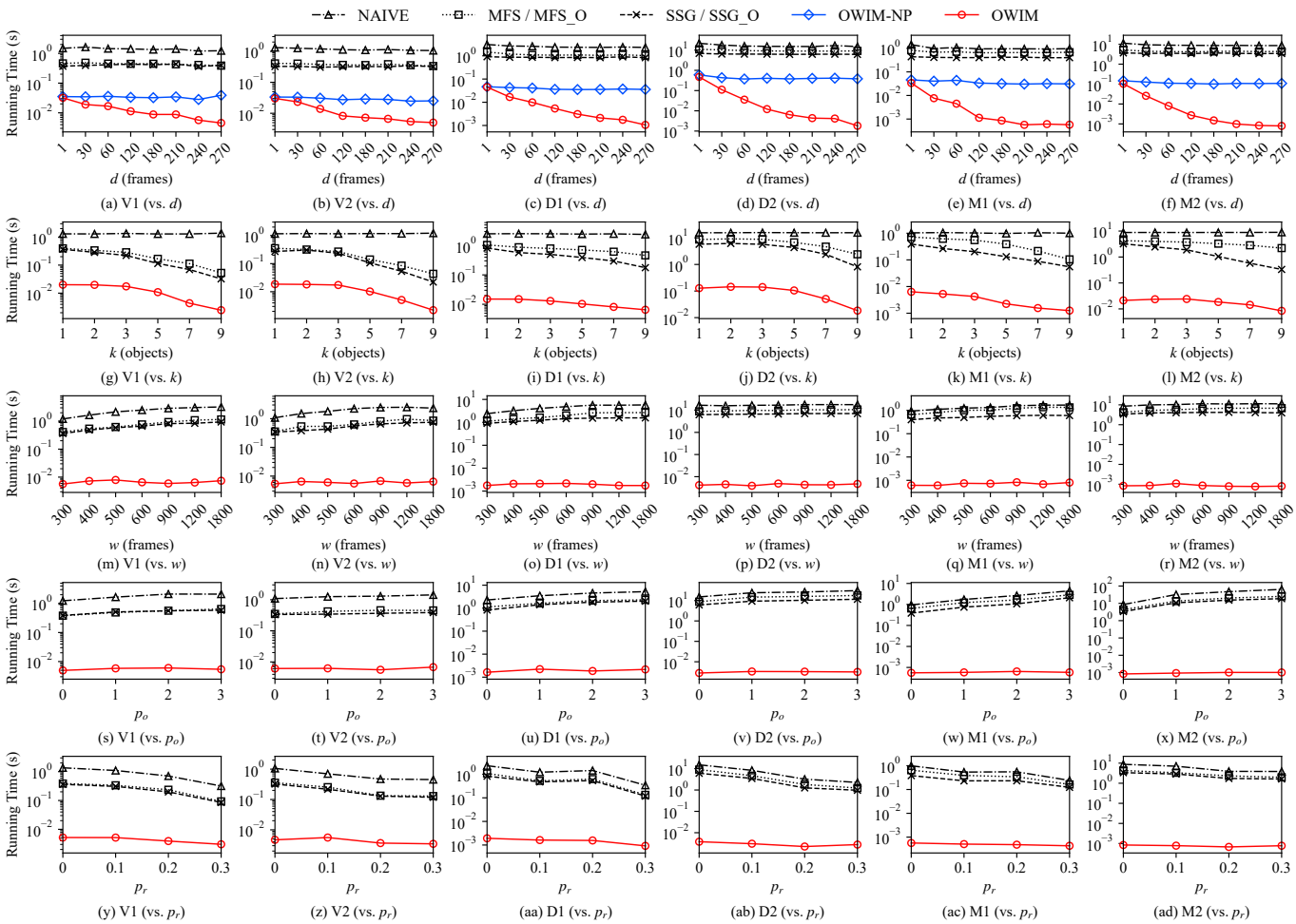


Fig. 7. Total maintenance time when varying d, k, w, p_o, and p_r: (a)–(f) varying d; (g)–(l) varying k with $d = 30$, where MFS and SSG are used when $k = 1$ and MFS_O and SSG_O when $k > 1$; (m)–(r) varying w; (s)–(x) varying p_o; (y)–(ad) varying p_r

and hardly changes with d. With $d = 1$, nothing is pruned and the two OWIM curves almost coincide, OWIM-NP taking only 4%–41% more time as it scans the whole table for the answers at every frame. Even then, OWIM is 11–35 times faster than SSG: with the same MCPs, SSG accesses all of its 290–6,882 states at every frame, whereas OWIM accesses only the entries that contain the new or the expiring frame, 2.7–13 times fewer, and each intersection is a bitwise AND. As d grows, the two OWIM curves quickly separate: at the default $d = 240$, SSG is 13–32 times slower than OWIM-NP, a gap unrelated to pruning, and OWIM-NP is 5–127 times slower than OWIM, a gap due to permanent pruning.

Varying k. Figs. 7(g)–(l) vary k with $d = 30$, using MFS and SSG for $k = 1$, where they have nothing to prune, and MFS_O and SSG_O for $k > 1$. NAIVE does not prune by k and stays unchanged, whereas the other three methods become faster as k grows. On most datasets, the answers change little for $k \leq 3$ (e.g., by only 15% on V1), and so does the time of OWIM; for $k \geq 5$, they decrease quickly, and OWIM becomes 2.3–8.4 times faster from $k = 1$ to $k = 9$. Over all k, OWIM is 10–152, 14–261, and 62–1,060 times faster than SSG_O, MFS_O, and NAIVE, respectively. SSG_O usually speeds up the most (by 4.7–12.3 times), narrowing the advantage of OWIM on M2 from 152 to 39 times; however, it misses more answers as k grows (96% on M2 at $k = 9$), so its time is biased low.

Varying w. In Figs. 7(m)–(r), with $d = 240$, the methods of [1] slow down by 6%–181% from $w = 300$ to $w = 1800$, since a longer window contains more MCPs (e.g., the states per frame grow from 1,245 to 3,923 on D1 and from 6,882 to 13,962 on M2). The time of OWIM stays almost unchanged, within a factor of 1.4 on each dataset: the answers per frame grow with w (e.g., from 4.2 to 24.4 on V1) but remain at most a few dozen, and the bitmaps grow from 5 to 29 words, which makes each intersection more expensive but hardly affects the total time. With $w = 1800$, OWIM is 117–4,954 times faster than SSG.

D. Effect of Data Characteristics

Figs. 7(s)–(ad) show the total maintenance time of all methods on the six videos as p_o and p_r vary, with the other parameters at their defaults.

Varying p_o. In Figs. 7(s)–(x), the methods of [1] slow down by a factor of 1.2–7.5 from $p_o = 0$ to $p_o = 3$, most on M1 and M2, which are captured by moving cameras (by a factor of 5.4

TABLE V
PER-FRAME LATENCY (μS) UNDER THE DEFAULT PARAMETERS

	SSG			OWIM		
Dataset	mean	p99	max	mean	p99	max
V1	198	557	623	3.0	10.8	13.9
V2	194	623	854	3.2	12.2	14.3
D1	767	2,609	3,236	1.4	4.4	12.6
D2	5,449	49,550	51,805	3.0	13.0	33.4
M1	334	1,053	1,220	0.4	1.0	4.4
M2	4,628	13,780	15,806	1.1	3.6	19.3
L1	6,042	29,254	121,251	2.6	38.6	482.3
L2	1,150	5,571	22,220	4.1	21.2	431.6
L3	1,532	9,380	26,099	7.7	52.7	499.6

and 5.6 for SSG). A reused identifier stands for several objects that appear one after another, so its frames in the window are scattered and it co-occurs with more objects; the window thus contains more MCPs, e.g., the states per frame on M2 grow from 6,839 to 32,766. The answers that satisfy the duration threshold, however, hardly change (0–4.8 per frame), so the time of OWIM grows by at most 32%; the more occlusions, the larger its advantage, and with $p_o = 3$ it is 57–18,371 times faster than SSG.

Varying p_r. In Figs. 7(y)–(ad), removing objects reduces both the objects per frame and the MCPs in the window, so all methods become faster. The methods of [1] benefit the most, their time dropping to 12%–48% of the original from $p_r = 0$ to $p_r = 0.3$ (e.g., the states per frame on D2 drop from 6,623 to 1,196), whereas the time of OWIM, which maintains only the answers, drops only to 48%–92%. The advantage of OWIM thus shrinks, yet with $p_r = 0.3$ it is still 28–2,104 times faster than SSG. Since the removed objects are drawn independently for each value of p_r, the curves are not strictly monotonic; e.g., on D1 the methods of [1] are slightly slower with $p_r = 0.2$ than with $p_r = 0.1$.

E. Per-frame Latency and Memory

Per-frame latency. Table V reports the mean, the 99th percentile, and the maximum of the per-frame latency of SSG and OWIM under the default parameters; MFS and NAIVE are slower than SSG and are omitted. OWIM takes 0.4–7.7 μs per frame on average, at most 53 μs at the 99th percentile, and less than 0.5 ms in the worst case, whereas SSG takes 0.19–6.0 ms, 0.56–49.5 ms, and up to 121 ms, respectively. At 30 frames per second, a frame must be processed within about 33 ms; on D2, state maintenance alone exceeds this budget on 103 frames (9%) for SSG and on 111 and 120 frames for MFS and NAIVE, whereas the slowest frame of OWIM there takes only 33 μs.

Fig. 8 shows the latency of every frame on the six datasets. The latency of SSG rises and falls with the number of states it maintains, with a correlation coefficient of 0.93–1.00 on all datasets except M1. On M1, which is captured by a moving camera, the latency of SSG changes by more than 2 times between consecutive frames on 11% of the frames (at most 1.5% on the other datasets), and the correlation coefficient drops to 0.48. The shape of the curves depends on the content of the video. In the later part of D2, more objects are in view, and the states maintained by SSG reach 57,726 at frame 1,114, which takes SSG 52 ms to process. On M2, the latency of SSG peaks at 15.8 ms at frame 204 and falls below 1 ms after frame 470. On V1 and V2, where objects stay long, SSG maintains at most 1,102 states and takes at most 0.85 ms per frame. The latency of OWIM stays within 0.1–33 μs on all six datasets; even in the last part of D2, where SSG is the slowest, OWIM takes at most 15 μs per frame. OWIM is faster than SSG on every frame of every dataset, and at least 10 times faster on over 98% of the frames.

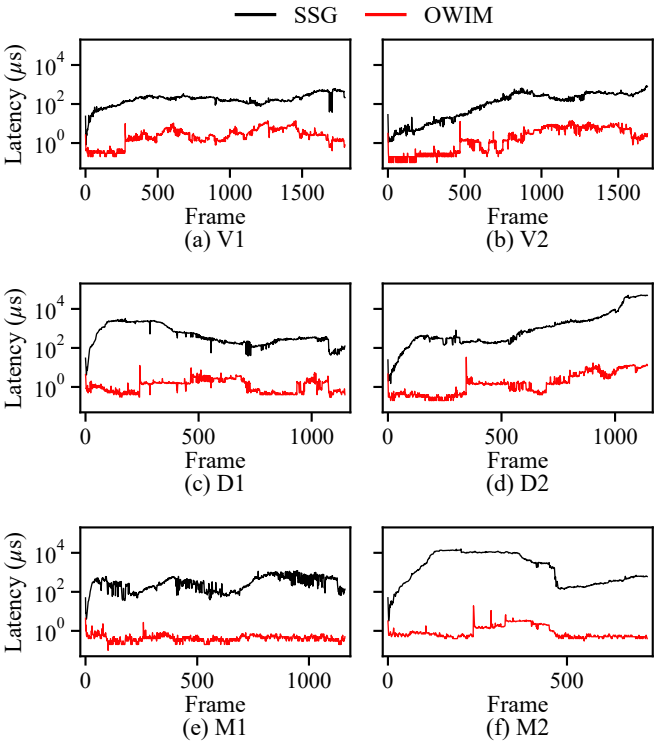


Fig. 8. Per-frame latency under the default parameters

Memory. Table VI reports the peak memory of each method during maintenance under the default parameters. To measure it, we periodically run a full garbage collection during maintenance and record the live heap of the JVM in excess of that after the input is loaded. OWIM takes only 39–57 KB on all datasets: its table holds only the answers, appearance bitmaps are kept only for the objects of the current window, and the bitmaps of objects that leave the window are recycled. The methods of [1] keep all MCPs of the window, so their memory grows with the number of states; SSG takes 1.06–68.07 MB, 22–1,235 times as much as OWIM. OWIM-NP maintains the same MCPs as SSG but takes 2.8–5.5 times less memory, since it represents a frame set by a w-bit bitmap, whereas the implementation of [1] keeps a per-frame list of frame numbers for each state.

VI. RELATED WORK

Query processing over videos. Much work on declarative video analytics reduces the cost of deep models with cheap

TABLE VI
PEAK MEMORY (MB) UNDER THE DEFAULT PARAMETERS

Dataset	NAIVE	MFS	SSG	OWIM-NP	OWIM
V1	0.664	1.036	1.064	0.280	0.048
V2	0.967	1.644	1.685	0.376	0.047
D1	2.063	3.362	3.550	0.845	0.045
D2	40.982	64.998	68.065	12.346	0.055
M1	1.361	1.973	2.124	0.758	0.038
M2	8.545	14.276	14.952	3.239	0.049

filters and specialized models [21], [22], probabilistic predicates [23], and query optimization [24]–[26]; other work supports richer predicates over the detected objects, such as spatial constraints [27], object tracks [28], compositional events [29], and complex event patterns over video streams [30]. These techniques are orthogonal to ours, which works on the object stream produced by detection and tracking. Closest to our work, Chen et al. [1] introduce co-occurrence queries over video feeds with two maintenance methods, MFS and SSG. A follow-up [2] returns the top-ranked windows of stored videos by the same co-occurrence score; its offline indexes over video partitions and its early termination do not apply to our continuous queries, which must report all answers at every frame as the frames arrive. Other follow-ups retrieve clips of stored videos under spatial and temporal constraints [3] and track groups of spatially close objects frame by frame [11]; both require spatial relationships among objects, whereas co-occurrence queries only require the objects to appear in the same frame.

Co-movement patterns. Co-occurrence queries resemble co-movement patterns, i.e., groups of objects that stay spatially close for a minimum duration, where proximity is determined either by clustering object locations at each time point for GPS trajectories, e.g., with DBSCAN [31]–[33], or by close entrance times at the same camera for trajectories across a road network [4], [5]. A closed swarm [31], whose time points may be non-consecutive, is an MCP when all objects of every frame form one cluster; its miner, ObjectGrowth, searches object by object and prunes by duration, like our computation for a single window, but mines a whole database offline. Later work mines swarms in sliding windows [32], detects co-movement patterns on streaming trajectories with distributed processing [33], and mines offline the groups that travel together along a route of cameras [4], [5]. In contrast, co-occurrence queries work on the object stream of a single camera and only require objects to appear in the same frame, with neither object locations nor clustering.

Object detection and tracking. Our object streams come from object detection and multi-object tracking, whose advances are orthogonal to this work: detectors have evolved from two-stage designs [34] to one-stage real-time [6], [14], [35] and transformer-based ones [7], [36], and most trackers follow tracking by detection, associating detections across frames by motion and appearance [8]–[10], [12], [37]. Occlusions and identity switches still fragment tracks; co-occurrence queries tolerate short occlusions, as an object set needs to co-occur in only d of the w frames, and fragmented tracks can be merged during video ingestion [38].

VII. CONCLUSION

We studied continuous co-occurrence queries over object streams from videos and proposed OWIM, which scans object by object, safely prunes candidates by both the duration and the size thresholds, and maintains exactly the answer set at every frame. With circularly encoded appearance bitmaps and an oldest-frame index, it processes, at each frame, only the entries that contain the expiring or the new frame. Experiments show that OWIM consistently outperforms the competitors.

REFERENCES

[1] Y. Chen, X. Yu, N. Koudas, and Z. Yu, "Evaluating temporal queries over video feeds," in *SIGMOD '21: International Conference on Management of Data, Virtual Event, China, June 20-25, 2021*, G. Li, Z. Li, S. Idreos, and D. Srivastava, Eds. ACM, 2021, pp. 287–299. [Online]. Available: https://doi.org/10.1145/3448016.3452803

[2] Y. Chen, X. Yu, and N. Koudas, "Ranked window query retrieval over video repositories," in *38th IEEE International Conference on Data Engineering, ICDE 2022, Kuala Lumpur, Malaysia, May 9-12, 2022*. IEEE, 2022, pp. 2776–2791. [Online]. Available: https://doi.org/10.1109/ICDE53745.2022.00253

[3] Y. Chen, N. Koudas, X. Yu, and Z. Yu, "Spatial and temporal constrained ranked retrieval over videos," *Proc. VLDB Endow.*, vol. 15, no. 11, pp. 3226–3239, 2022. [Online]. Available: https://www.vldb.org/pvldb/vol15/p3226-yu.pdf

[4] D. Zhang, T. Ma, J. Hu, Y. Bei, K.-L. Tan, and G. Chen, "Co-movement pattern mining from videos," *Proc. VLDB Endow.*, vol. 17, no. 3, pp. 604–616, 2023. [Online]. Available: https://www.vldb.org/pvldb/vol17/p604-zhang.pdf

[5] Y. Bei, T. Ma, D. Zhang, S. Wu, K.-L. Tan, and G. Chen, "Mining platoon patterns from traffic videos," *Proc. VLDB Endow.*, vol. 18, no. 6, pp. 1839–1851, 2025. [Online]. Available: https://www.vldb.org/pvldb/vol18/p1839-bei.pdf

[6] Y. Tian, Q. Ye, and D. S. Doermann, "YOLOv12: Attention-centric real-time object detectors," in *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2025, NeurIPS 2025, San Diego, CA, USA, December 2-7, 2025 / Mexico City, Mexico, November 30 - December 5, 2025*, D. Belgrave, C. Zhang, L. N. Montoya, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, N. Chen, I. V. M. Ruíz, and A. Loaiza-Bonilla, Eds., 2025.

[7] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, "DETRs beat YOLOs on real-time object detection," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2024, Seattle, WA, USA, June 16-22, 2024*. IEEE, 2024, pp. 16 965–16 974. [Online]. Available: https://doi.org/10.1109/CVPR52733.2024.01605

[8] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-object tracking by associating every detection box," in *Computer Vision - ECCV 2022 - 17th European Conference, Tel Aviv, Israel, October 23-27, 2022, Proceedings, Part XXII*, ser. Lecture Notes in Computer Science, S. Avidan, G. J. Brostow, M. Cissé, G. M. Farinella, and T. Hassner, Eds., vol. 13682. Springer, 2022, pp. 1–21. [Online]. Available: https://doi.org/10.1007/978-3-031-20047-2_1

[9] J. Cao, J. Pang, X. Weng, R. Khirodkar, and K. Kitani, "Observation-centric SORT: rethinking SORT for robust multi-object tracking," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2023, Vancouver, BC, Canada, June 17-24, 2023*. IEEE, 2023, pp. 9686–9696. [Online]. Available: https://doi.org/10.1109/CVPR52729.2023.00934

[10] Y. Du, Z. Zhao, Y. Song, Y. Zhao, F. Su, T. Gong, and H. Meng, "StrongSORT: Make DeepSORT great again," *IEEE Trans. Multim.*, vol. 25, pp. 8725–8737, 2023. [Online]. Available: https://doi.org/10.1109/TMM.2023.3240881

[11] H. Yoon, D. Choi, and Y. D. Chung, "Group tracking for video monitoring systems: A spatio-temporal query processing approach," *IEEE Access*, vol. 11, pp. 19 969–19 987, 2023. [Online]. Available: https://doi.org/10.1109/ACCESS.2023.3249190

[12] N. Wojke, A. Bewley, and D. Paulus, "Simple online and realtime tracking with a deep association metric," in *2017 IEEE International Conference on Image Processing, ICIP 2017, Beijing, China, September 17-20, 2017*. IEEE, 2017, pp. 3645–3649. [Online]. Available: https://doi.org/10.1109/ICIP.2017.8296962

[13] Z. Zhang, J. Zhang, B. Wang, and X. Yang, "Continuous co-occurrence query over object streams from videos (long version)," Technical report, 2026. [Online]. Available: https://github.com/zhangzhongshuai/Continuous-Co-Occurrence-Query/blob/main/Continuous_Co-Occurrence_Query_over_Object_Streams_from_Videos_long_version.pdf

[14] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," *CoRR*, vol. abs/1804.02767, 2018. [Online]. Available: http://arxiv.org/abs/1804.02767

[15] B. Haynes, A. Mazumdar, M. Balazinska, L. Ceze, and A. Cheung, "Visual Road: A video data management benchmark," in *Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019, Amsterdam, The Netherlands, June 30 - July 5, 2019*. ACM, 2019, pp. 972–987. [Online]. Available: https://doi.org/10.1145/3299869.3324955

[16] S. Lyu, M. Chang, D. Du, L. Wen, H. Qi, Y. Li, Y. Wei, L. Ke, T. Hu, M. Del Coco *et al.*, "UA-DETRAC 2017: Report of AVSS2017 & IWT4S challenge on advanced traffic monitoring," in *14th IEEE International Conference on Advanced Video and Signal Based Surveillance, AVSS 2017, Lecce, Italy, August 29 - September 1, 2017*. IEEE, 2017, pp. 1–7. [Online]. Available: https://doi.org/10.1109/AVSS.2017.8078560

[17] A. Milan, L. Leal-Taixé, I. D. Reid, S. Roth, and K. Schindler, "MOT16: A benchmark for multi-object tracking," *CoRR*, vol. abs/1603.00831, 2016. [Online]. Available: http://arxiv.org/abs/1603.00831

[18] "YouTube video," https://www.youtube.com/watch?v=wqctLW0Hb_0.

[19] "YouTube video," https://www.youtube.com/watch?v=e_WBuBqS9h8.

[20] "YouTube video," https://www.youtube.com/watch?v=Inbd-YOdPUE.

[21] D. Kang, J. Emmons, F. Abuzaid, P. Bailis, and M. Zaharia, "NoScope: Optimizing neural network queries over video at scale," *Proc. VLDB Endow.*, vol. 10, no. 11, pp. 1586–1597, 2017. [Online]. Available: http://www.vldb.org/pvldb/vol10/p1586-kang.pdf

[22] D. Kang, P. Bailis, and M. Zaharia, "BlazeIt: Optimizing declarative aggregation and limit queries for neural network-based video analytics," *Proc. VLDB Endow.*, vol. 13, no. 4, pp. 533–546, 2019. [Online]. Available: http://www.vldb.org/pvldb/vol13/p533-kang.pdf

[23] Y. Lu, A. Chowdhery, S. Kandula, and S. Chaudhuri, "Accelerating machine learning inference with probabilistic predicates," in *Proceedings of the 2018 International Conference on Management of Data, SIGMOD Conference 2018, Houston, TX, USA, June 10-15, 2018*, G. Das, C. M. Jermaine, and P. A. Bernstein, Eds. ACM, 2018, pp. 1493–1508. [Online]. Available: https://doi.org/10.1145/3183713.3183751

[24] J. Cao, K. Sarkar, R. Hadidi, J. Arulraj, and H. Kim, "FiGO: Fine-grained query optimization in video analytics," in *SIGMOD '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12 - 17, 2022*, Z. G. Ives, A. Bonifati, and A. E. Abbadi, Eds. ACM, 2022, pp. 559–572. [Online]. Available: https://doi.org/10.1145/3514221.3517857

[25] J. Bang, G. T. Kakkar, P. Chunduri, S. Mitra, and J. Arulraj, "SEIDEN: revisiting query processing in video database systems," *Proc. VLDB Endow.*, vol. 16, no. 9, pp. 2289–2301, 2023. [Online]. Available: https://www.vldb.org/pvldb/vol16/p2289-kakkar.pdf

[26] D. Chao, Y. Chen, N. Koudas, and X. Yu, "Optimizing video queries with declarative clues," *Proc. VLDB Endow.*, vol. 17, no. 11, pp. 3256–3268, 2024. [Online]. Available: https://www.vldb.org/pvldb/vol17/p3256-koudas.pdf

[27] N. Koudas, R. Li, and I. Xarchakos, "Video monitoring queries," in *36th IEEE International Conference on Data Engineering, ICDE 2020, Dallas, TX, USA, April 20-24, 2020*. IEEE, 2020, pp. 1285–1296. [Online]. Available: https://doi.org/10.1109/ICDE48307.2020.00115

[28] F. Bastani, S. He, A. Balasingam, K. Gopalakrishnan, M. Alizadeh, H. Balakrishnan, M. J. Cafarella, T. Kraska, and S. Madden, "MIRIS: fast object track queries in video," in *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference 2020, online conference [Portland, OR, USA], June 14-19, 2020*, D. Maier, R. Pottinger, A. Doan, W. Tan, A. Alawini, and H. Q. Ngo, Eds. ACM, 2020, pp. 1907–1921. [Online]. Available: https://doi.org/10.1145/3318464.3389692

[29] E. Zhang, M. Daum, D. He, B. Haynes, R. Krishna, and M. Balazinska, "EQUI-VOCAL: synthesizing queries for compositional video events from limited user interactions," *Proc. VLDB Endow.*, vol. 16, no. 11, pp. 2714–2727, 2023. [Online]. Available: https://www.vldb.org/pvldb/vol16/p2714-zhang.pdf

[30] C. Han, C. Chang, S. Srivastava, Y. Lu, and E. Lo, "Scalable complex event processing on video streams," *Proc. ACM Manag. Data*, vol. 3, no. 3, pp. 214:1–214:29, 2025. [Online]. Available: https://doi.org/10.1145/3725419

[31] Z. Li, B. Ding, J. Han, and R. Kays, "Swarm: Mining relaxed temporal moving object clusters," *Proc. VLDB Endow.*, vol. 3, no. 1, pp. 723–734, 2010. [Online]. Available: https://vldb.org/pvldb/vol3/R65.pdf

[32] A. Bhushan, U. Bellur, K. Sharma, S. Deshpande, and N. L. Sarda, "Mining swarm patterns in sliding windows over moving object data streams," in *Proceedings of the 25th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems, GIS 2017, Redondo Beach, CA, USA, November 7-10, 2017*, E. G. Hoel, S. D. Newsam, S. Ravada, R. Tamassia, and G. Trajcevski, Eds. ACM, 2017, pp. 60:1–60:4. [Online]. Available: https://doi.org/10.1145/3139958.3139988

[33] L. Chen, Y. Gao, Z. Fang, X. Miao, C. S. Jensen, and C. Guo, "Real-time distributed co-movement pattern detection on streaming trajectories," *Proc. VLDB Endow.*, vol. 12, no. 10, pp. 1208–1220, 2019. [Online]. Available: http://www.vldb.org/pvldb/vol12/p1208-chen.pdf

[34] S. Ren, K. He, R. B. Girshick, and J. Sun, "Faster R-CNN: towards real-time object detection with region proposal networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137–1149, 2017. [Online]. Available: https://doi.org/10.1109/TPAMI.2016.2577031

[35] J. Redmon, S. K. Divvala, R. B. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*. IEEE Computer Society, 2016, pp. 779–788. [Online]. Available: https://doi.org/10.1109/CVPR.2016.91

[36] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Computer Vision - ECCV 2020 - 16th European Conference, Glasgow, UK, August 23-28, 2020, Proceedings, Part I*, ser. Lecture Notes in Computer Science, A. Vedaldi, H. Bischof, T. Brox, and J. Frahm, Eds., vol. 12346. Springer, 2020, pp. 213–229. [Online]. Available: https://doi.org/10.1007/978-3-030-58452-8_13

[37] A. Bewley, Z. Ge, L. Ott, F. T. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *2016 IEEE International Conference on Image Processing, ICIP 2016, Phoenix, AZ, USA, September 25-28, 2016*. IEEE, 2016, pp. 3464–3468. [Online]. Available: https://doi.org/10.1109/ICIP.2016.7533003

[38] D. Chao, Y. Chen, N. Koudas, and X. Yu, "Track merging for effective video query processing," in *39th IEEE International Conference on Data Engineering, ICDE 2023, Anaheim, CA, USA, April 3-7, 2023*. IEEE, 2023, pp. 164–176. [Online]. Available: https://doi.org/10.1109/ICDE55515.2023.00020